



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 05

NOMBRE COMPLETO: Hernández Domínguez Luis Carlos

N° de Cuenta: 320182668

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 04

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 28/09/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1. Ejercicios

Ejercicio: Importar el modelo de un carro

1.- Importar su modelo de coche propio dentro del escenario a una escala adecuada.

2.- Importar sus 4 llantas y acomodarlas jerárquicamente, agregar el mismo valor de rotación a las llantas para que al presionar puedan rotar hacia adelante y hacia atrás.

3.- Importar el cofre del coche, acomodarlo jerárquicamente y agregar la rotación para poder abrir y cerrar.

4.- Agregar traslación con teclado para que pueda avanzar y retroceder de forma independiente.

Bloques de código generado

```
262 // K traslación hacia atrás
263 // L traslación al frente
264 // X abrir cofre
265 // C cerrar cofre
266 // V rotar llantas hacia el frente
267 // B rotar llantas hacia atrás
268 //Carro
269 color = glm::vec3(0.0f, 1.0f, 1.0f);
270 model = glm::mat4(1.0);
271 model = glm::translate(model, glm::vec3(0.0f, 5.7f, 10.0f));
272 model = glm::scale(model, glm::vec3(5.0f, 5.0f, 5.0f));
273 model = glm::translate(model, glm::vec3(mainWindow.getarticulacion6(), 0.0f, 0.0f));
274 model = glm::translate(model, glm::vec3(-mainWindow.getarticulacion5(), 0.0f, 0.0f));
275 modelaux = model;
276 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
277 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
278 carro.RenderModel();

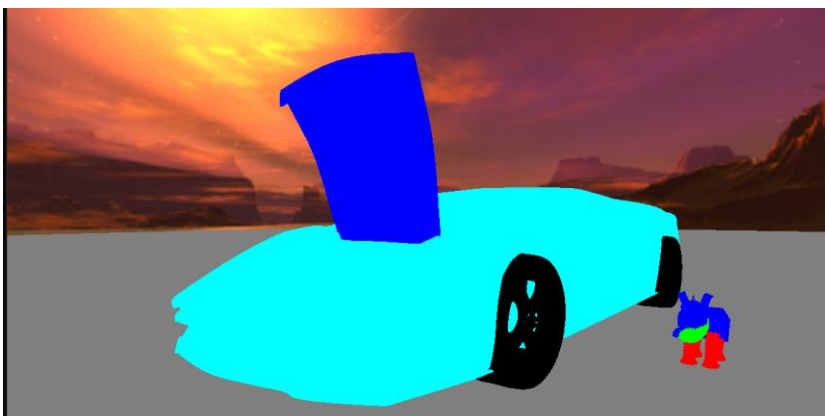
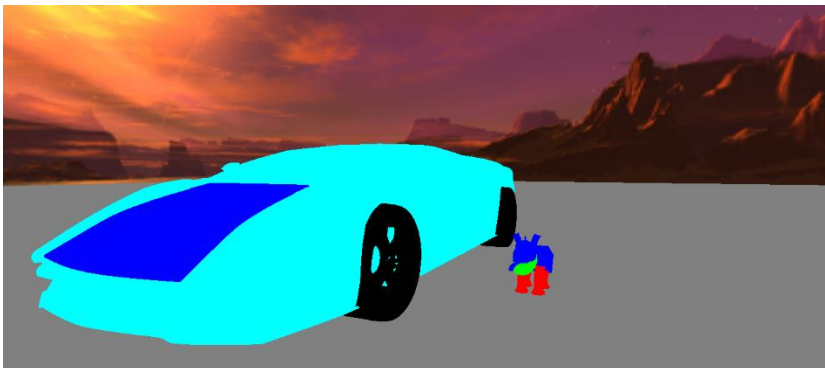
280 //llanta 1
281 model = modelaux;
282 color = glm::vec3(0.0f, 0.0f, 0.0f);
283 model = glm::translate(model, glm::vec3(2.75f, -0.8f, -2.2f));
284 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, -1.0f));
285 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(0.0f, 0.0f, 1.0f));
286 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
287 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
288 llanta.RenderModel();
289
290 //llanta 2
291 model = modelaux;
292 color = glm::vec3(0.0f, 0.0f, 0.0f);
293 model = glm::translate(model, glm::vec3(-2.9f, -0.8f, -2.2f));
294 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, -1.0f));
295 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(0.0f, 0.0f, 1.0f));
296 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
297 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
298 llanta.RenderModel();
```

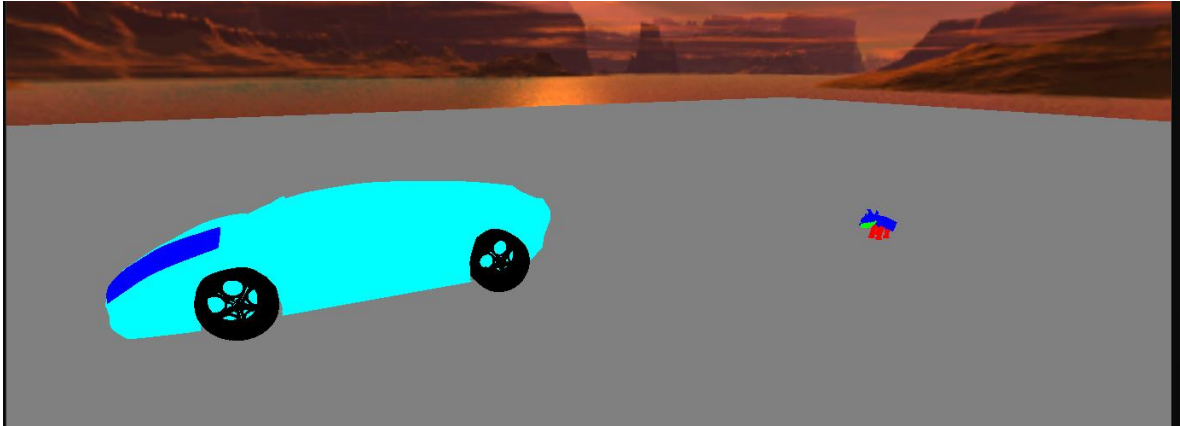
```

300 //llanta 3
301 model = modelaux;
302 color = glm::vec3(0.0f, 0.0f, 0.0f);
303 model = glm::translate(model, glm::vec3(2.75f, -0.785f, 1.5f));
304 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
305 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, 1.0f));
306 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(0.0f, 0.0f, -1.0f));
307 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
308 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
309 llanta.RenderModel();
310
311 //llanta 4
312 model = modelaux;
313 color = glm::vec3(0.0f, 0.0f, 0.0f);
314 model = glm::translate(model, glm::vec3(-2.75f, -0.785f, 1.5f));
315 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
316 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, 1.0f));
317 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(0.0f, 0.0f, -1.0f));
318 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
319 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
320 llanta.RenderModel();
321
322 //cofre
323 model = modelaux;
324 color = glm::vec3(0.0f, 0.0f, 1.0f);
325 model = glm::translate(model, glm::vec3(3.0f, 0.1f, -0.4f));
326 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 0.0f, 1.0f));
327 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
328 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
329 cofre.RenderModel();
330

```

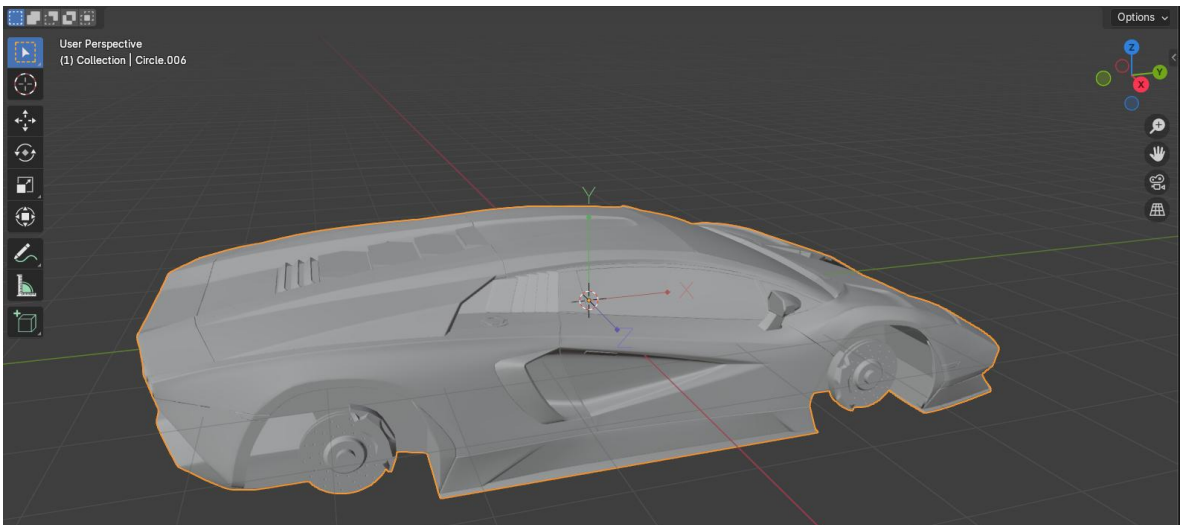
Ejecución



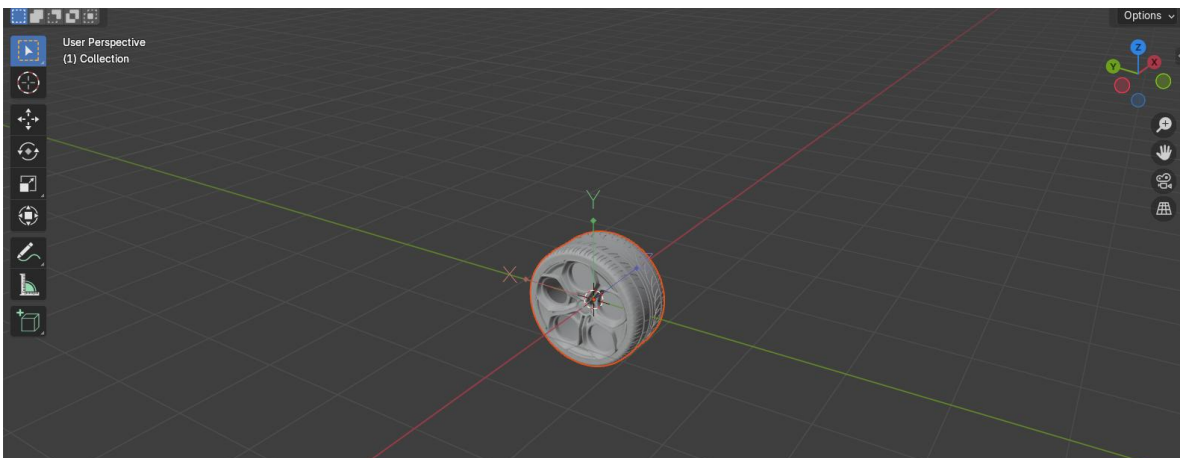


Modelos

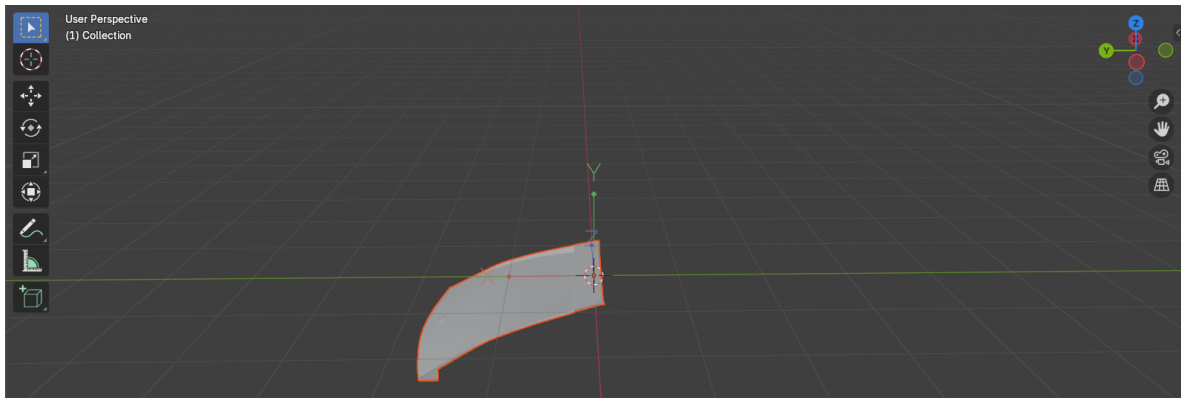
Auto sin llantas ni cofre



Llanta



Cofre



Para resolver este problema primero fue necesario separar el cofre y las llantas del modelo original y guardarlo para importarlo con OpenGL. Posteriormente, se obtienen los modelos de la llanta y el cofre y se guardan por separado para importarlos.

Una vez que se tuvieron los tres modelos fue necesario escalarlos para que tuvieran un tamaño relativo al Goddard realizado en clase. Se mantuvo a Goddard para mostrar la diferencia de tamaño entre los dos modelos. Finalmente, para implementar la traslación y rotación fue necesario declarar nuevas teclas para poder avanzar el carro, abrir/cerrar el cofre y rotar las llantas

2. Problemas presentados.

No se presentó ningún problema a la hora de ejecutar código.

3.- Conclusión:

- Esta práctica me pareció bastante entretenida, sobre todo por el uso de Blender que es una herramienta que he aprendido a usar recientemente en teoría. Y ser capaz de integrar modelos realizados allí con OpenGL me pareció algo bastante interesante.
- Los conceptos y explicaciones dadas en clase fueron correctas y suficientes para la resolución de esta práctica.
- En conclusión, esta práctica me pareció tener un nivel de dificultad adecuado y hace un buen trabajo al introducir el modelo jerárquico.